**LAPORAN TUGAS EKSPLORASI MANDIRI**

Apache Spark dan NoSQL — Mahadata / Big Data (IF25-40402)

# Replikasi Parsial Strategi Paralel Segmentasi Citra Seed-Based pada Apache Spark

**(Chen et al., 2021)**




**M Farisi Suyitno 123140152**

**FAKULTAS TEKNOLOGI INDUSTRI**

**PROGRAM STUDI TEKNIK INFORMATIKA**

**INSTITUT TEKNOLOGI SUMATERA**

**2026**

## A. Ringkasan Jurnal

**Sitasi.** Chen, F., Wang, N., Yu, B., Qin, Y., & Wang, L. (2021). A strategy of parallel seed-based image segmentation algorithms for handling massive image tiles over the Spark platform. *Remote Sensing, 13*(10), 1969. https://doi.org/10.3390/rs13101969

**Reputasi.** Artikel terbit di *Remote Sensing* (MDPI), akses terbuka (CC BY), memiliki DOI, dan mencantumkan tanggal diterima 20 April 2021, disetujui 14 Mei 2021, serta terbit 18 Mei 2021 (memenuhi syarat 2019 ke atas). Kuartil SJR: [isi kuartil SJR dari scimagojr.com].

**Masalah.** Citra penginderaan jauh berukuran sangat besar sehingga perlu diproses pada klaster. Platform big data seperti Spark hanya menyediakan sedikit algoritma pengolahan citra, sedangkan algoritma sekuensial yang dijalankan langsung pada tile menghasilkan objek tidak lengkap di tepi tile dan volume komunikasi besar pada tahap shuffle.

**Dataset dan lingkungan artikel.** Sepuluh citra Sentinel-2 RGB tahun 2020 dari Copernicus Open Access Hub, masing-masing dipotong 4096 x 4096 piksel, dengan tile 256 sampai 2048 piksel. Klaster memiliki tujuh node fisik. Eksperimen waktu memakai empat executor dengan empat core dan memori 2 GB per executor, dengan driver 15 GB.

**Metode.** Dua algoritma berbasis seed (region growing dan watershed) dijalankan dalam enam langkah: pretreatment citra, segmentasi tile, deteksi ukuran buffer, sintesis auxiliary band, pemrosesan area border, dan integrasi tile. Dua gagasan utama menekan volume komunikasi: (1) ukuran buffer dideteksi otomatis memakai *accumulator* Spark, bukan ditebak besar; dan (2) tiga auxiliary band (garis border, kategori, arah) digabung menjadi satu band yang menggantikan band seed.

**Hasil utama.** Pada tile 256 dan kepadatan seed level 1, rata-rata F-measure dengan dan tanpa pemrosesan border adalah 95,2% dan 92,8% (region growing, seluruh citra), 90,2% dan 84,4% (region growing, area border), 99,8% dan 99,1% (watershed, seluruh citra), serta 98,1% dan 91,1% (watershed, area border). Waktu eksekusi umumnya bertambah seiring ukuran buffer dan jumlah auxiliary band, dan strategi yang diusulkan berjalan sekitar dua kali lebih cepat daripada strategi tanpa pengurangan volume komunikasi.

**Alasan memilih.** Artikel memuat metode yang dapat dijalankan, datanya terbuka (Sentinel-2), parameternya cukup rinci, dan konsep Spark yang dipakai (RDD, *lazy evaluation*, action, persist, accumulator, shuffle) sesuai materi kuliah. Skalanya dapat diperkecil menjadi satu mesin dengan mode lokal.

## B. Tahapan Implementasi dan Eksperimen

### B.1 Tingkat replikasi

Replikasi **parsial**: seluruh alur enam langkah dan kedua algoritma direplikasi, tetapi pada citra 1024 x 1024 piksel (bukan 4096 x 4096), 3 citra, dan Spark mode lokal (bukan klaster tujuh node). Rasio jumlah tile per sisi dipertahankan: tile 64, 128, 256, 512 pada citra 1024 setara dengan tile 256, 512, 1024, 2048 pada citra 4096.

### B.2 Rancangan eksperimen

| Unsur | Isi |
|---|---|
| Tujuan | Menguji apakah pemrosesan area border memperbaiki akurasi, dan apakah satu auxiliary band serta buffer otomatis menurunkan waktu dan volume shuffle. |
| Data | 3 citra RGB 1024 x 1024 (synthetic (citra sintetis bergaya lanskap)). |
| Lingkungan | Spark mode lokal pada satu mesin gratis (laptop atau Google Colab). |

| Unsur | Isi |
|---|---|
| Langkah | Pretreatment, referensi sekuensial, pipeline Spark, evaluasi F-measure, pengukuran waktu. |
| Keluaran dibandingkan | F-measure dengan dan tanpa border processing (seluruh citra dan area border), waktu 1 vs 3 auxiliary band, buffer otomatis vs manual, volume shuffle. |

**B.3 Lingkungan dan versi perangkat lunak**

| Komponen | Versi |
|---|---|
| Python | 3.13.16 |
| NumPy / SciPy / scikit-image | 2.5.3 / 1.18.1 / 0.26.0 |
| PySpark | tidak dipakai (backend LocalContext) |
| Mode eksekusi | LocalContext (tanpa Spark) |
| Jumlah CPU mesin | 2 |
| Tanggal eksperimen | 2026-10-13 13:23 |

**B.4 Langkah demi langkah**

- **Praproses.** RGB diubah ke abu-abu dengan Persamaan (1), lalu median filter berradius 2 piksel. Seed region growing adalah puncak lokal maksimum dan minimum pada jendela 33, 27, 21, 15, dan 9 (level 1 sampai 5). Pita spektral watershed adalah gradien (rank.gradient dengan disk radius 1) dan seed-nya piksel dengan gradien < 2, 3, 4, 5, dan 6. Seed diberi id unik, dan id itu menjadi label objek.
- **Referensi.** Segmentasi sekuensial seluruh citra dengan algoritma yang sama. Region growing memakai implementasi sendiri (antrean prioritas, mengikuti Adams & Bischof, 1994); watershed memakai skimage.segmentation.watershed (Vincent & Soille, 1991).
- **Tile menjadi RDD.** Citra dipotong menjadi tile T x T dan di-parallelize dengan satu partisi per tile.
- **Segmentasi tile dan deteksi buffer.** Tiap tile disegmentasi sendiri. Objek yang menyentuh sisi dalam tile dicatat sebagai objek border, dan kedalaman terbesarnya ke arah dalam tile dikirim ke *accumulator* bertipe maksimum. Karena evaluasi Spark malas, persist() dan count() dipanggil sebelum nilai accumulator dibaca.
- **Auxiliary band tunggal.** Satu band int32 per tile: 0 untuk piksel inner (dikunci), id seed untuk seed di area border, dan -1 untuk piksel area border tanpa seed. Varian pembanding mengirim seed, garis border, kategori, dan arah sebagai band terpisah (11 byte per piksel dibanding 5 byte, termasuk pita spektral).
- **Shuffle.** Tiap tile mengirim potongan yang beririsan dengan jendela ber-buffer ke delapan tetangganya lewat flatMap dan groupByKey. Jumlah byte yang dikirim ke tetangga dihitung dengan accumulator penjumlah sebagai ukuran volume komunikasi.
- **Border area processing.** Jendela ber-buffer dirakit, lalu algoritma dijalankan ulang hanya pada area border dengan seed di dalamnya. Hasil dipotong ke tile dan hanya piksel area border yang diganti.
- **Evaluasi.** F-measure berbasis objek (Persamaan 2 sampai 6, $\alpha = 0{,}5$) terhadap referensi sekuensial, untuk seluruh citra dan untuk area border, yaitu piksel objek yang menyentuh sisi tile pada hasil tanpa processing.
- **Waktu.** Waktu dinding seluruh pipeline dengan time.perf_counter, satu kali pemanasan, dan 1 ulangan. Buffer manual dipilih 20%, 40%, 60%, 80%, dan 100% ukuran tile (padanan 100 sampai 500 piksel pada tile 512 di artikel).

### B.5 Perbedaan terhadap artikel

| Aspek | Artikel | Replikasi ini | Alasan |
|---|---|---|---|
| Ukuran citra | 4096 x 4096, 10 citra | 1024 x 1024, 3 citra | Sumber daya terbatas dan batas tenggat |
| Data | Sentinel-2 Copernicus | synthetic (citra sintetis bergaya lanskap) | Akun Copernicus dan unduhan tidak tersedia pada lingkungan pengembangan |
| Klaster | 7 node, 4 executor x 4 core | LocalContext, satu mesin | Gratis dan ringan |
| Kode | Tidak dilampirkan | Dibangun ulang dari deskripsi | Rincian border processing hanya garis besar |
| Auxiliary band | Garis border, kategori, arah | Mask + seed dalam satu band | Disederhanakan dengan hasil setara |
| Penentuan seed watershed | Ambang gradien | rank.gradient disk radius 1 (asumsi) | Fungsi gradien tidak dirinci |

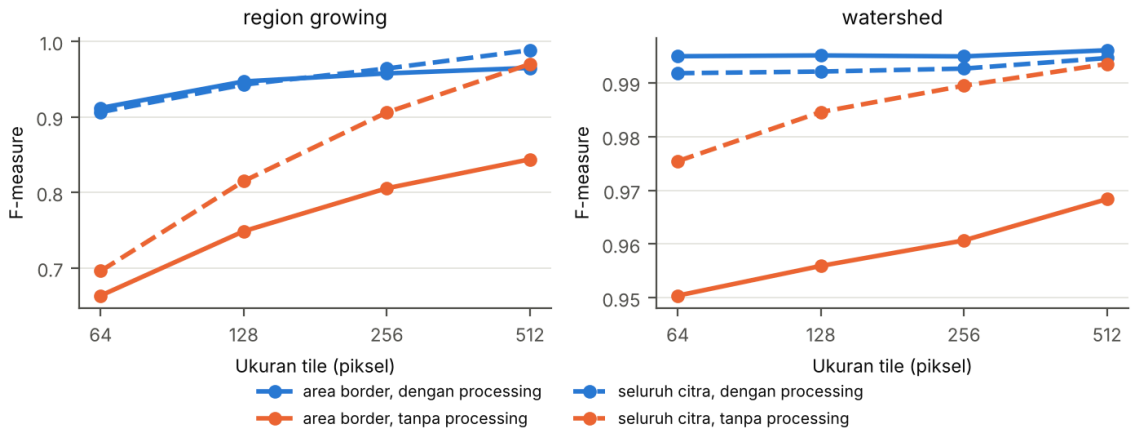
## C. Hasil Pengujian dan Analisis

Catatan sumber data: Hasil ini dihasilkan dengan LocalContext (tanpa Spark). Akurasi dan volume shuffle tidak bergantung pada backend karena algoritmanya deterministik, tetapi waktu eksekusi bukan waktu Spark. Citra yang dipakai sintetis, bukan Sentinel-2.

### C.1 Akurasi

Tabel 1 memuat rata-rata F-measure atas 3 citra dan lima level seed.

| Algoritma | Tile | Seluruh citra: dengan | Seluruh citra: tanpa | Area border: dengan | Area border: tanpa | Buffer otomatis rata-rata |
|---|---|---|---|---|---|---|
| region growing | 64 | 90.6% | 69.6% | 91.2% | 66.3% | 64 |
| region growing | 128 | 94.3% | 81.5% | 94.7% | 74.9% | 128 |
| region growing | 256 | 96.4% | 90.6% | 95.8% | 80.6% | 240 |
| region growing | 512 | 98.8% | 97.0% | 96.5% | 84.4% | 309 |
| watershed | 64 | 99.2% | 97.5% | 99.5% | 95.0% | 64 |
| watershed | 128 | 99.2% | 98.5% | 99.5% | 95.6% | 128 |
| watershed | 256 | 99.3% | 98.9% | 99.5% | 96.1% | 256 |
| watershed | 512 | 99.5% | 99.3% | 99.6% | 96.8% | 462 |




Gambar 1. F-measure rata-rata menurut ukuran tile. Garis biru: dengan border area processing; oranye: tanpa.

Tabel 2 membandingkan dengan angka artikel (tile 256 pada citra 4096, 16 x 16 tile, level seed 1) terhadap kasus replikasi terdekat: tile 64 pada citra 1024 (16 x 16 tile), level seed 1.

| Kasus | Artikel dengan | Artikel tanpa | Replikasi dengan | Replikasi tanpa |
|---|---|---|---|---|
| region growing, seluruh citra | 95.2% | 92.8% | 83.6% | 56.9% |
| region growing, area border | 90.2% | 84.4% | 83.5% | 55.4% |
| watershed, seluruh citra | 99.8% | 99.1% | 98.5% | 94.8% |
| watershed, area border | 98.1% | 91.1% | 99.0% | 87.8% |

**Analisis akurasi.**

- **Region growing.** F-measure area border naik dari 76.5% menjadi 94.5% (+18.0 poin) setelah border area processing, sedangkan untuk seluruh citra 84.7% menjadi 95.1% (+10.4 poin). Hasil dengan processing tidak lebih rendah pada 100% kombinasi (citra, level, tile). Selisih area border mengecil dari +24.9 poin pada tile 64 menjadi +12.1 poin pada tile 512. F-measure seluruh citra dengan processing naik dari 91.9% (level 1) ke 98.2% (level 5).
- **Watershed.** F-measure area border naik dari 95.9% menjadi 99.5% (+3.6 poin) setelah border area processing, sedangkan untuk seluruh citra 98.6% menjadi 99.3% (+0.7 poin). Hasil dengan processing tidak lebih rendah pada 100% kombinasi (citra, level, tile). Selisih area border mengecil dari +4.4 poin pada tile 64 menjadi +2.8 poin pada tile 512. F-measure seluruh citra dengan processing naik dari 98.7% (level 1) ke 99.7% (level 5).

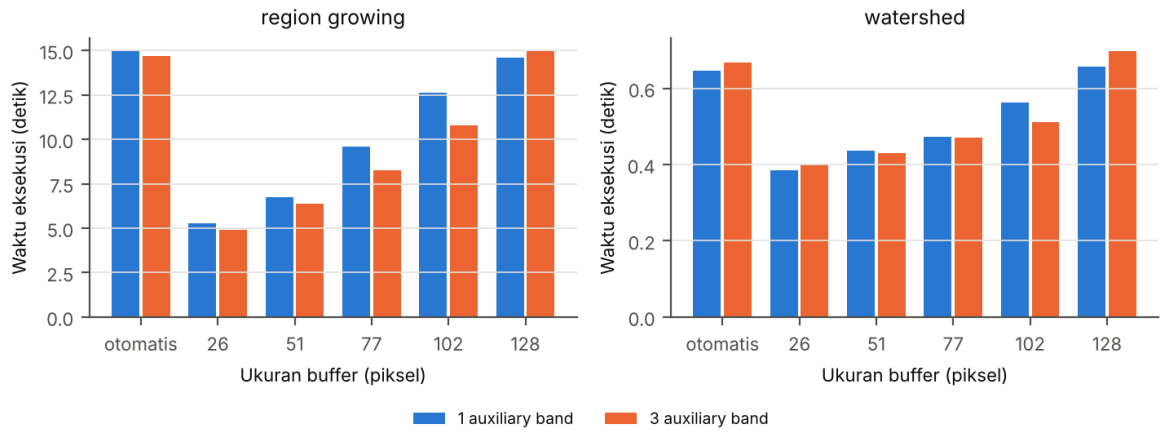
Selisih pada seluruh citra jauh lebih kecil daripada pada area border karena objek border hanya sebagian kecil piksel; pada tile kecil porsinya lebih besar sehingga manfaat pemrosesan border lebih terlihat. Angka replikasi tidak identik dengan artikel karena citra, implementasi region growing, dan definisi area border berbeda, sehingga yang dibandingkan adalah arah dan besar perubahannya.

### C.2 Efisiensi: auxiliary band, buffer, dan volume shuffle

Tabel 3 dihitung pada tile 128 piksel dan level seed 3.

| Algoritma | Band | Mode buffer | Buffer rata-rata (piksel) | Waktu rata-rata (detik) | Volume shuffle (MB) |
|---|---|---|---|---|---|
| region growing | 1 | otomatis | 128 | 14.96 | 34.4 |

| Algoritma | Band | Mode buffer | Buffer rata-rata (piksel) | Waktu rata-rata (detik) | Volume shuffle (MB) |
|---|---|---|---|---|---|
| region growing | 1 | manual (rata-rata 5 ukuran) | 77 | 9.77 | 18.1 |
| region growing | 3 | otomatis | 128 | 14.71 | 75.7 |
| region growing | 3 | manual (rata-rata 5 ukuran) | 77 | 9.05 | 39.7 |
| watershed | 1 | otomatis | 128 | 0.65 | 34.4 |
| watershed | 1 | manual (rata-rata 5 ukuran) | 77 | 0.50 | 18.1 |
| watershed | 3 | otomatis | 128 | 0.67 | 75.7 |
| watershed | 3 | manual (rata-rata 5 ukuran) | 77 | 0.50 | 39.7 |




Gambar 2. Waktu eksekusi menurut ukuran buffer untuk 1 dan 3 auxiliary band (rata-rata atas citra).

- **Region growing.** Tiga auxiliary band membutuhkan waktu rata-rata 0.94 kali waktu satu band (10.00 s dibanding 10.64 s) dan mengirim 2.20 kali volume shuffle. Dengan satu band, waktu buffer manual terkecil (26 piksel) adalah 5.28 s dan terbesar (128 piksel) 14.61 s. Buffer otomatis rata-rata 128 piksel memakai 14.96 s, dibanding 14.61 s pada buffer manual terdekat (128 piksel).
- **Watershed.** Tiga auxiliary band membutuhkan waktu rata-rata 1.01 kali waktu satu band (0.53 s dibanding 0.53 s) dan mengirim 2.20 kali volume shuffle. Dengan satu band, waktu buffer manual terkecil (26 piksel) adalah 0.38 s dan terbesar (128 piksel) 0.66 s. Buffer otomatis rata-rata 128 piksel memakai 0.65 s, dibanding 0.66 s pada buffer manual terdekat (128 piksel).

Karena backend lokal, waktu di atas mencerminkan komputasi Python pada satu proses dan tidak memuat biaya shuffle jaringan Spark; kolom volume shuffle tetap valid.

**Keterbatasan.** (1) Skala citra dan mesin jauh lebih kecil daripada artikel. (2) Waktu pada satu mesin dipengaruhi proses lain; pengulangan terbatas. (3) Buffer otomatis rata-rata 100% ukuran tile, mendekati ukuran tile karena objek besar menyentuh sisi tile, sehingga pengurangan volume komunikasi lebih kecil daripada klaim artikel. (4) Definisi area border dan aturan border processing dirancang sendiri karena artikel tidak merinci algoritmanya. (5) Waktu shuffle tidak diukur terpisah.

### D. Kendala dan Refleksi

- **Artikel tanpa kode dan rincian parsial.** Aturan penentuan mask, seed, dan batas pemrosesan ulang di area border dirancang sendiri dari Gambar 2 dan 3 serta teks artikel; diuji dengan membandingkan

satu tile (harus identik dengan sekuensial) dan memastikan border processing tidak menurunkan akurasi.

- **Region growing tidak tersedia di pustaka.** Implementasi Python murni memerlukan sekitar delapan detik per citra 1024 x 1024, sehingga ukuran citra dikecilkan.
- **Data.** Unduhan Copernicus memerlukan akun, sehingga disediakan skrip pengambil potongan Sentinel-2 dari AWS Earth Search (tanpa akun) dan generator citra sintetis untuk uji tanpa internet.
- **Evaluasi malas Spark.** Nilai accumulator kosong sebelum ada action, sehingga count() dan persist() perlu dipanggil sebelum membaca ukuran buffer. Accumulator di dalam transformasi dapat terhitung ganda jika tugas diulang; jenis maksimum aman, sedangkan penghitung byte hanya perkiraan.
- **Lingkungan pengembangan.** Kode dikembangkan dengan bantuan AI pada lingkungan yang tidak dapat memasang PySpark, sehingga logika diuji lewat LocalContext yang meniru API RDD. Pengukuran waktu Spark perlu dijalankan ulang pada mesin yang memiliki PySpark.

**Jika diulang:** memakai citra Sentinel-2 asli untuk seluruh skenario, menentukan buffer per tile (bukan satu nilai global) seperti usulan artikel, mengukur waktu tahap shuffle dari Spark UI, dan mencoba klaster kecil di cloud untuk melihat biaya jaringan yang sebenarnya.

### E. Tautan Kode

Repositori: IcniP/123140152_mahadata

### Pernyataan Penggunaan Generative AI

Generative AI (Claude, Anthropic) digunakan untuk membantu menulis kode awal (segpipe.py, run_experiments.py, get_data.py, uji unit), menyusun skrip pembuat laporan, dan menyusun draf laporan ini. Penulis bertanggung jawab atas kebenaran isi dan menjalankan serta memahami kode tersebut.

**Daftar Pustaka**

Adams, R., & Bischof, L. (1994). Seeded region growing. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 16*, 641–647.

Chen, F., Wang, N., Yu, B., Qin, Y., & Wang, L. (2021). A strategy of parallel seed-based image segmentation algorithms for handling massive image tiles over the Spark platform. *Remote Sensing, 13*(10), 1969. https://doi.org/10.3390/rs13101969

van der Walt, S., Schönberger, J. L., Nunez-Iglesias, J., Boulogne, F., Warner, J. D., Yager, N., Gouillart, E., & Yu, T. (2014). scikit-image: Image processing in Python. *PeerJ, 2*, e453. https://doi.org/10.7717/peerj.453

Vincent, L., & Soille, P. (1991). Watersheds in digital spaces: An efficient algorithm based on immersion simulations. *IEEE Transactions on Pattern Analysis and Machine Intelligence, 13*, 583–598.

Zaharia, M., Xin, R. S., Wendell, P., Das, T., Armbrust, M., Dave, A., Meng, X., Rosen, J., Venkataraman, S., Franklin, M. J., Ghodsi, A., Gonzalez, J., Shenker, S., & Stoica, I. (2016). Apache Spark: A unified engine for big data processing. *Communications of the ACM, 59*, 56–65.

Zhang, X., Feng, X., Xiao, P., He, G., & Zhu, L. (2015). Segmentation quality evaluation using region-based precision and recall measures for remote sensing images. *ISPRS Journal of Photogrammetry and Remote Sensing, 102*, 73–84.